

Credit Card Approval Prediction

Phase 7: Project Documentation

Complete Technical Reference

SmartBridge / SkillWallet Virtual Internship Program

Team Member	Role
Tupakula Rishmitha Sri	Team Lead
Tejomai Kondapureddy	Member
Korrapolu Lahari	Member
Saragada Tirupathi Venkata Mohan Reddy	Member
Vijay Kanth Kalvakuntla	Member

1. Project Overview

The Credit Card Approval Prediction System automates the credit card approval decision using machine learning. A predictive model is trained on historical applicant data — income, employment history, education, family situation, and credit behaviour — to evaluate new applications and instantly return an Approved or Rejected decision with a confidence score, through a Flask web application.

2. Repository Structure

```
Credit_Card_Approval_Prediction/
├── Dataset/
│   ├── application_record.csv
│   └── credit_record.csv
├── Flask/
│   ├── app.py
│   ├── templates/index.html
│   └── model/
│       ├── model.pkl
│       ├── model_columns.pkl
│       ├── scaler.pkl
│       └── encoders.pkl
└── Project_Documentation/
    ├── 1.Brainstorming_&_Ideation/
    ├── 2.Requirement_Analysis/
    ├── 3.Project_Design_Phase/
    ├── 4.Project_Planning_Phase/
    ├── 5.Project_Development_Phase/
    ├── 6.Project_Testing/
    └── 7.Project_Documentation/
```

- ■■■ 8.Project_Demonstration/
 - Training/
 - ■■■ Credit_Card_Approval_Model_Training.ipynb
 - requirements.txt
 - README.md
 - .gitignore

3. Dataset

application_record.csv — one row per applicant: ID, CODE_GENDER, FLAG_OWN_CAR, FLAG_OWN_REALTY, CNT_CHILDREN, AMT_INCOME_TOTAL, NAME_INCOME_TYPE, NAME_EDUCATION_TYPE, NAME_FAMILY_STATUS, NAME_HOUSING_TYPE, DAYS_BIRTH, DAYS_EMPLOYED, FLAG_MOBIL, FLAG_WORK_PHONE, FLAG_PHONE, FLAG_EMAIL, OCCUPATION_TYPE, CNT_FAM_MEMBERS.

credit_record.csv — one row per applicant per month: ID, MONTHS_BALANCE, STATUS (0 = 1-29 days past due, 1-5 = increasingly overdue/bad debt, C = paid off, X = no active loan that month).

4. Preprocessing Pipeline

- Drop duplicate applicant rows (drop_duplicates on all attribute columns).
- Convert DAYS_BIRTH / DAYS_EMPLOYED to positive values with abs().
- Label-encode CODE_GENDER, FLAG_OWN_CAR, FLAG_OWN_REALTY, NAME_INCOME_TYPE, NAME_EDUCATION_TYPE, NAME_FAMILY_STATUS, NAME_HOUSING_TYPE.
- Convert multi-class STATUS to binary STATUS_BIN via a to_binary() function.
- Aggregate credit_record.csv per applicant ID into OPEN_MONTH, END_MONTHS, WINDOW, EMI_PAID_OFF, EMI_PASTDUES, NUMBER_OF_LOANS, and TARGET.
- Merge application_record.csv with the aggregated credit history on ID.

5. Machine Learning Models

5.1 Logistic Regression

A linear model that estimates approval probability as a weighted sum of features passed through a sigmoid function. Fast, interpretable, and useful as a baseline — each coefficient shows how strongly a feature pushes the decision. Configuration: max_iter=1000, class_weight='balanced'.

5.2 Decision Tree

Splits the data repeatedly on the feature/threshold that best separates Approved from Rejected, forming a tree of yes/no questions. Captures non-linear rules without manual engineering. Configuration: max_depth=8, class_weight='balanced'.

5.3 Random Forest

An ensemble of 300 Decision Trees, each trained on a random bootstrap sample and random feature subset; predictions are combined by majority vote. Reduces the overfitting risk of a single tree. Configuration: n_estimators=300, max_depth=10, class_weight='balanced'.

5.4 XGBoost

A gradient boosting ensemble where each new tree corrects the errors of the trees before it. Typically the strongest performer on structured, tabular financial data. Configuration: n_estimators=300, max_depth=5, learning_rate=0.08, scale_pos_weight=ratio of rejected to approved applicants.

6. Model Comparison Results

Model	Accuracy	F1-score
Logistic Regression	0.902	0.944
Decision Tree	0.909	0.949
Random Forest (deployed)	0.939	0.966
XGBoost	0.921	0.956

7. Flask Application

A single-page Flask application. The `/` route (GET) renders the empty applicant form. The `/predict` route (POST) collects form values, encodes categorical fields with the saved `LabelEncoders`, scales numeric fields with the saved `StandardScaler`, calls `model.predict()` / `predict_proba()`, and re-renders the same page with the Approved/Rejected result and confidence %.

8. Deployment (IBM Watson Machine Learning — Optional)

- Push the repository to GitHub.
- Deploy the Flask application on IBM Cloud.
- Register the trained model with IBM Watson Machine Learning.
- Store `model.pkl` and preprocessing objects in IBM Cloud Object Storage.
- The Flask app calls the deployed Watson ML REST endpoint instead of the local model file, enabling scalable, real-time inference in production.